

SmartSpend

AI-Powered Expense Management System

A Spring Boot backend system that helps users track expenses, manage budgets, set saving goals, and analyze receipts using AI.





The Problem

Managing personal expenses manually can be difficult. Users may forget their spending, lose receipts, exceed their budgets, or struggle to understand where their money goes.

Our Solution

SmartSpend provides one system to:

- Track manual expenses and receipts.
- Analyze receipt images automatically using AI.
- Monitor monthly or yearly budgets.
- Track saving goals.
- Analyze spending habits.
- Send budget status notifications by email.

Technologies Used

Backend Java + Spring Boot

Used to build REST APIs, business logic, validation, and database operations.

Database MySQL + JPA

Used to store users, expenses, budgets, saving goals, categories, and receipts.

Gemini AI API

Used to analyze receipt images and automatically extract:

- Store name
- Total amount
- Purchase date
- Category

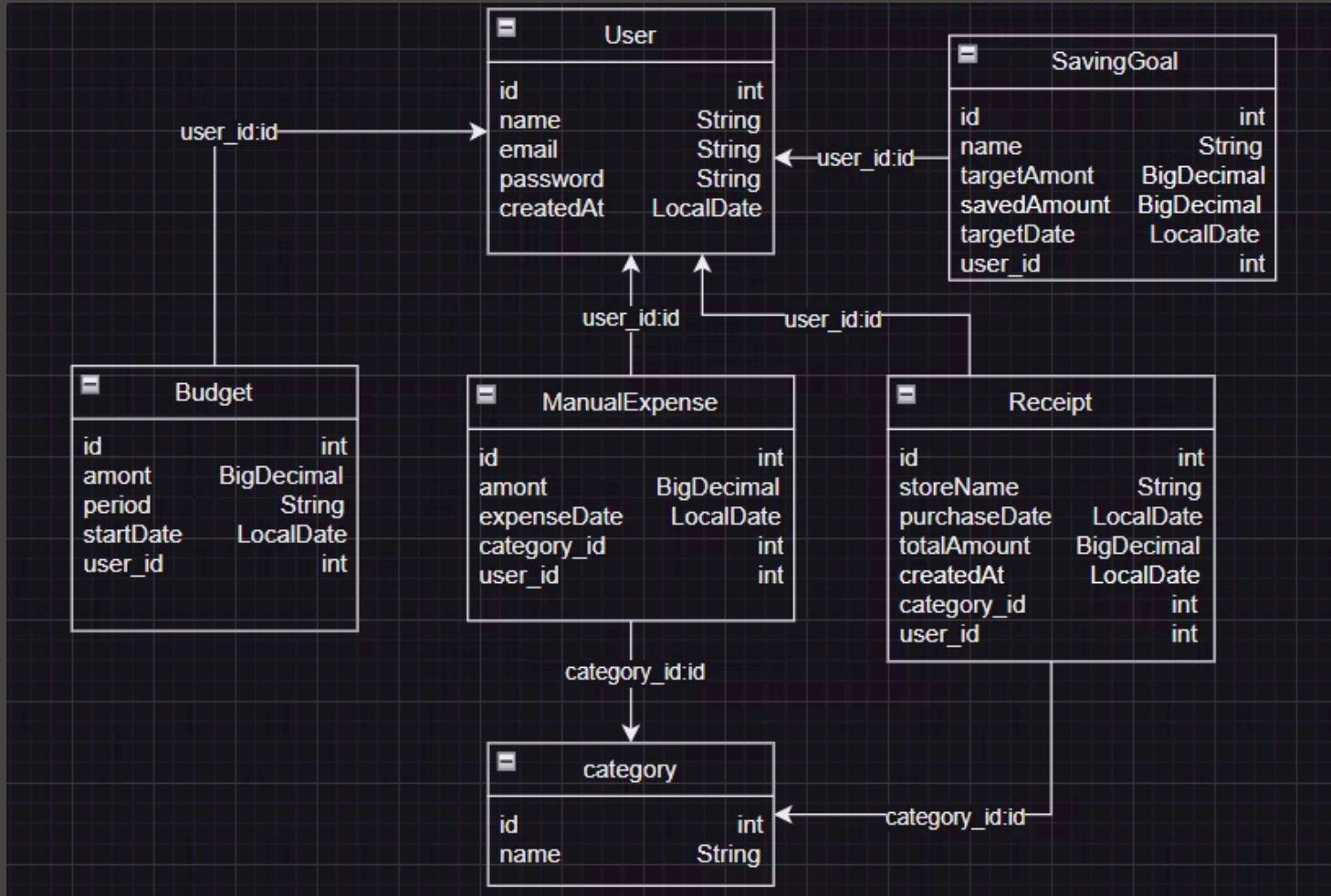
Email Service Spring Mail / Gmail SMTP

Used to send budget status notifications to users by email.

Testing Postman

Used to test all APIs and upload receipt images.

Classes



SmartSpend Extra Endpoints

1. Total Spending
2. Total Spending Between Dates
3. Spending By Category
4. Highest Spending Category
5. Highest Expense
6. Average Daily Spending
7. Monthly Spending
8. Compare Monthly Spending
9. Remaining Budget
10. Budget Usage Percentage
11. Check Budget Exceeded
12. Forecast Budget Spending
13. Add Money To Saving Goal
14. Saving Goal Progress
15. Required Saving Per Month
16. Send Budget Status Email

AI Receipt Analysis

SmartSpend converts receipt images into structured expense data automatically.



Receipt Image



Gemini AI Analysis

Extracted Data

```
{  
  "storeName": "Katana  
Sushi",  
  "totalAmount": 153.71,  
  "purchaseDate": "2018-05-  
11",  
  "category": "Restaurants"  
}
```

Receipt Image → Base64 → Gemini API → Structured JSON → Saved to Database

Why It's Useful

- No manual receipt entry
- Automatically extracts important information
- Automatically classifies the expense
- Saves the result directly as a Receipt

Send Budget Status Email

Endpoint

POST /budget/send-budget-status/{user_id}

Example

POST /budget/send-budget-status/2

Possible Responses

- Budget status email sent successfully
- User or budget not found

What It Does

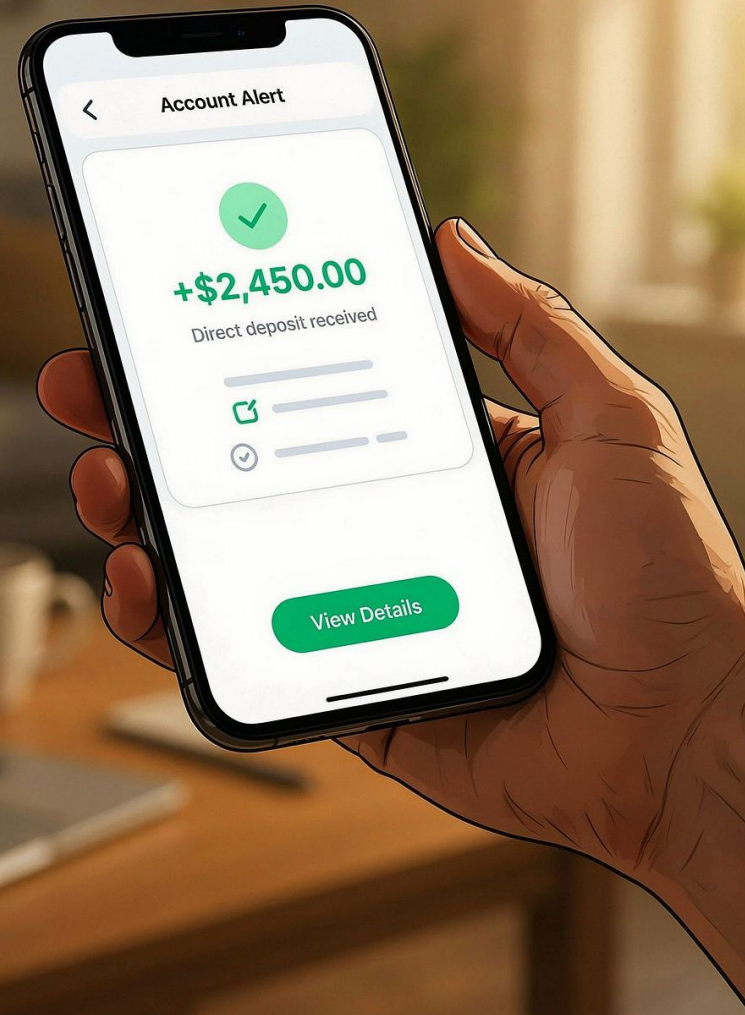
Checks the user's current budget status and sends an email.

If the user exceeded the budget, the email contains the exceeded amount.

If the budget is still safe, the email contains the remaining amount.

Why It's Useful

Allows users to receive important financial updates outside the application.



Forecast Budget Spending

Endpoint

GET /budget/forecast/{user_id}

Example

GET /budget/forecast/2

Possible Responses

- Forecasted spending: 6450.00
- User or budget not found

What It Does

Calculates the user's average daily spending and predicts the total spending by the end of the current budget cycle.

Why It's Useful

Helps the user identify possible overspending before the budget period ends.



Check Budget Exceeded

Endpoint

GET /budget/check-exceeded/{user_id}

Example

GET /budget/check-exceeded/2

Possible Responses

- Budget not exceeded.
Remaining: 2500.00
- Budget exceeded by: 500.00

What It Does

Compares the user's current spending with the budget amount.

Why It's Useful

Immediately shows whether the user is still within the budget or has exceeded it.





Saving Goal Progress

Endpoint

GET /saving-goal/progress/{id}

Example

GET /saving-goal/progress/2

Possible Responses

- Progress: 37.50% - Remaining: 5000.00
- Saving goal not found

What It Does

Calculates the percentage completed for a saving goal and the remaining amount.

Why It's Useful

Helps users easily monitor how close they are to reaching their financial goal.



Required Saving Per Month

Endpoint

GET /saving-goal/required-saving-per-month/{id}

Example

GET /saving-goal/required-saving-per-month/2

Possible Responses

- Required saving per month:
1000.00
- Saving goal not found

What It Does

Calculates how much money the user should save every month to reach the target before the target date.

Why It's Useful

Turns a large saving goal into a clear monthly saving plan.



Total Spending Between Dates

Endpoint

```
GET /user/total-spending-  
between-  
dates/{user_id}/{startDate}/{end  
Date}
```

Example

```
GET /user/total-spending-  
between-dates/2/2026-09-  
01/2026-09-30
```

Possible Responses

- Total spending between dates: 1250.00
- End date must be after start date
- User not found

What It Does

Combines receipt expenses and manual expenses between two selected dates.

Why It's Useful

Allows users to analyze spending during a specific period.

Highest Spending Category

Endpoint

GET /user/highest-spending-category/{user_id}

Example

GET /user/highest-spending-category/2

Possible Responses

- Shopping - Total: 1200.00
- No spending found
- User not found

What It Does

Calculates spending for every category and identifies the category where the user spent the most money.

Why It's Useful

Helps users understand where most of their money is going.





Compare Monthly Spending

Endpoint

```
GET /user/compare-monthly-  
spending/{user_id}/{firstYear}/{fir  
stMonth}/{secondYear}/{second  
Month}
```

Example

```
GET /user/compare-monthly-  
spending/2/2026/8/2026/9
```

Possible Responses

- Spending increased by: 700.00
- Spending decreased by: 400.00
- Spending is the same

What It Does

Calculates total spending for two different months and compares the results.

Why It's Useful

Allows users to see whether their spending habits are improving or getting worse.